

CSI 4107 Assignment 1

Group Members and Tasks:

Andrew Pham 300226985

Task: Part 1 (Preprocessing)

Caleb Bailey 300281841

Task: Part 3 (Testing) and Report

Lucas Gavura 300310069

Task: Part 2 (Indexing)

Martin Patrouchev 300286634

Task: Part 3 (Ranking and Retrieval)

Functionality:

Part 1: Preprocessing

Algorithm/flow

In our preprocessing file, we aim to prepare the corpus for information retrieval through a variety of techniques. The first is tokenization. In order to properly tokenize our words, we used the `word_tokenize` function from NLTK. The second step is stopwords removal, which uses `stopwords.txt` to find all words that would not be helpful to our queries. The function `remove_extras` provides this functionality. It checks to see if any of the given tokens exist within `stopwords.txt`, and if so, remove them from consideration for future queries. We also used stemming to help merge different word forms.

Data structures

- Lists
- Counters

Part 2: Indexing

Algorithm/flow

This section creates an inverted index from the already preprocessed documents from part 1. This section uses a dictionary to map each term with their document IDs, so that each term is associated with a document ID and a term frequency. We do this by using python's defaultdict.

Data structures

- Dictionary
- Counters

Part 3: Ranking and Retrieval:

Algorithm/flow

For the query retrieval, we used the same preprocessing logic from part 1 to preprocess the queries in order to stay consistent. Then, for a given query we use the inverted index defined in part 2 to retrieve a set of documents containing the query term(s). Next, we acquire the tf_idf vectors, which are combined with the magnitude to obtain our cosine similarities. This then tells us how similar a document is to the given query.

For the testing and results section, we were able to reuse much of the same logic as the first section in part 3. We went through each query and calculated the cosine similarity

to any relevant documents. From there, we just saved the top 100 results to the Results.txt file.

Data structures:

- Tuples
- Vectors

Optimizations

- Our preprocessing step had our stopwords stored as a set, allowing for $O(1)$ access time
- We used sparse vectors instead of dense vectors in our part 3, allowing for increased performance.
- Default dict is more efficient than standard dictionaries, as it decreases the number of checks needed when updating.

Sample 100 tokens from vocabulary:

['microstructur', 'develop', 'human', 'newborn', 'cerebr', 'white', 'matter', 'assess', 'vivo', 'diffus', 'tensor', 'magnet', 'reson', 'imag', '.', 'alter', 'architectur', 'brain', 'affect', 'cortic', 'result', 'function', 'disabl', 'line', 'scan', 'diffusion-weight', '(', 'mri', ')', 'sequenc', 'analysi', 'appli', 'measur', 'appar', 'coeffici', ',', 'calcul', 'rel', 'anisotropi', 'delin', 'three-dimension', 'fiber', 'preterm', '=', '17', 'full-term', 'infant', '7', 'effect', 'prematur', 'earli', 'gestat', '10', 'studi', 'term', 'central', 'mean', '28', 'wk', '1.8', 'microm2/m', 'decreas', '1.2', 'posterior', 'limb', 'intern', 'capsul', 'versu', '1.1', 'closer', 'birth', 'absolut', 'valu', '1.4', '+/-', '0.24', '1.15', '0.09', '0.016', 'area', 'compar', '10.9', '0.6', '22.9', '3.0', '%', '0.001', ';', '24.0', '4.44', '33.1', '0.006', 'nonmyelin', 'corpu', 'callosum', 'visibl', 'mark', 'differ', 'organ', 'data']

The vocabulary is 44632 tokens long

Query results:

9 Q0 44265107 1 0.496223 run1
9 Q0 24700152 2 0.291477 run1
9 Q0 6477536 3 0.162562 run1
9 Q0 14647747 4 0.143368 run1
9 Q0 19511011 5 0.131301 run1
9 Q0 9056874 6 0.127149 run1
9 Q0 32787042 7 0.125880 run1
9 Q0 25182647 8 0.119724 run1
9 Q0 26105746 9 0.117139 run1
9 Q0 2437807 10 0.114077 run1

11 Q0 25510546 1 0.229119 run1
11 Q0 1887056 2 0.190195 run1
11 Q0 13780287 3 0.188273 run1
11 Q0 5107861 4 0.173242 run1
11 Q0 4399311 5 0.173007 run1
11 Q0 29459383 6 0.156165 run1
11 Q0 16712164 7 0.151719 run1
11 Q0 2251426 8 0.139345 run1
11 Q0 32587939 9 0.133094 run1
11 Q0 13958154 10 0.128089 run1

For query 9, we can observe that the top result has a mid level cosine similarity, at 0.49, going down to 0.11 at the 10th rank.

When we look at query 11, we can see that it starts at a lower cosine similarity, and levels out to a 0.12 by rank 10.

How to Run Program

- In an appropriate python environment, make sure you have the scifact dataset files installed and placed in a folder named scifact.
- From here, run the preprocessing script to tokenize and remove stopwords from the documents
- Next, run the indexing script to build the inverted index.
- [test.py](#) already calls the previous steps, so simply running this script should produce any required output for the assignment.

Mean Average Precision (MAP)

Full text:

```
$ ./trec_eval -m map qrels/test/qrels.test run_full.txt
map          all    0.5400
```

Titles:

```
$ ./trec_eval -m map qrels/test/qrels.test run_title.txt
map          all    0.4029
```

The mean average precision for our regular model which included the full text was 0.5400. This is quite good, and means that the model we created is reliable and functional.

When comparing this to the run that only included only the titles, we can see that the latter has a worse performance at 0.4029. This is due to the fact that the title only run has far fewer tokens, and many query words may not appear in the titles at all.